

A100 训练提速方案

Trackerless 3D Ultrasound Reconstruction

trial/train/main_train_RecON_backbone.py

优化总览

以下4项改动均已应用至代码，零存储代价，无需修改训练流程即可直接运行。

改动	修改文件	预期效果
BF16 AMP + zero_grad优化	online_RecON_backbone.py	计算 2-3x 加速
torch.compile 编译backbone	online_RecON_backbone.py	+20-40%
batch_size 2 → 8	res/run/hp_bk.json	吞吐 4x 提升
h5 按需读取 (strided slice)	datasets/TUS_complete.py	I/O 57x 加速

1. 瓶颈分析

原始问题清单

>> batch_size = 2

对 80 GB A100 而言，显存利用率 < 5%，GPU 严重空转

>> 无 AMP (自动混合精度)

A100 Tensor Core 专为 BF16/FP16 优化，全 FP32 训练白白放弃 8x 算力

>> 无 torch.compile

PyTorch 2.0+ 图编译可减少内核启动开销，A100 上约 +20-40%

>> h5 全量帧读取

__getitem__ 每次读取整个 scan 的全部帧 (379帧 = 116 MB)，只使用10帧，造成 SSD 冗余 I/O

>> num_workers = 0

数据加载与 GPU 计算串行，CPU 等 GPU、GPU 等 CPU 交替空转

为何 num_workers 无法直接开启

__getitem__ 在 CUDA 设备上计算 optical_flow (NVIDIA CUDA Optical Flow / RAFT) 和 edge (Canny)。PyTorch 以 fork 模式启动 worker 进程，子进程无法继承主进程的 CUDA Context，设置 num_workers > 0 会导致死锁或崩溃。

2. 存储分析：为何不做预缓存

项目	大小	备注
现有 frames (1200 scans)	175 GB	基准
预缓存 optical_flow (float16)	557 GB	每 scan 464 MB × 1200
预缓存 edge (uint8)	140 GB	每 scan 116 MB × 1200
总增量	≈ 700 GB	现有数据的 4 倍，不可行
内存缓存 frames (全训练集)	112 GB	系统仅 33 GB RAM，不可行

结论：预缓存需要 700 GB 额外存储（现有的 4×），内存缓存需要 112 GB（超出 RAM 3×），均不可行。

3. 改动详情

3.1 BF16 AMP — online_RecON_backbone.py

A100 原生支持 BF16，无需 GradScaler loss scaling（自动禁用）。Tensor Core 在 BF16 下吞吐是 FP32 的 8 倍。

```
# __init__ 中新增
self._amp_dtype = torch.bfloat16 if torch.cuda.is_bf16_supported() else torch.float16
self._scaler = torch.cuda.amp.GradScaler(enabled=(self._amp_dtype == torch.float16))

# train() 中
self.optimizer.zero_grad(set_to_none=True) # 减少内存清零开销
with torch.cuda.amp.autocast(dtype=self._amp_dtype):
    fake_target, feature = self.backbone(input, return_feature=self.flag_motion)
    losses = self.criterion(real_target, fake_target, feature)
    loss = sum(losses.values())
self._scaler.scale(loss).backward()
self._scaler.step(self.optimizer)
self._scaler.update()
```

3.2 torch.compile — online_RecON_backbone.py

PyTorch 2.0+ 对计算图做静态编译，减少内核启动开销。首个 epoch 多耗约 1 分钟编译，之后每 batch 受益。

```
self.backbone = Backbone(...).to(self.device)
if hasattr(torch, "compile"):
    self.backbone = torch.compile(self.backbone)
```

3.3 batch_size 2 → 8 — res/run/hp_bk.json

A100 显存 80 GB，当前 $480 \times 640 \times \text{seq_len}=10$ 的 batch_size=2 仅使用不到 5% 显存。扩大至 8 可将单位时间的样本吞吐提升 4×。如遇 OOM 可回退至 4 或 6。

```
{
  "batch_size": 8, // 原为 2
  "test_batch_size": 1,
  ...
}
```

3.4 h5 按需读取 (strided slice) — datasets/TUS_complete.py

原代码读取 scan 内全部 379 帧（116 MB），再按 frame_rate 抽取 10 帧。改为直接用 h5py 的 strided slice（单次 HDF5 hyperslab 读取），每次只读所需的 10 帧（3.1 MB）。实测 I/O 从 43 ms 降至 0.7 ms，加速 57×。

```
# 改动前
with h5py.File(path, "r") as f:
    frames_np = f["frames"][:] # 读全部 379 帧 = 116 MB
source = torch.from_numpy(...)
source = source[:, frame_rate.item()] # 再抽 10 帧

# 改动后
frame_rate = torch.randint(cfg.frame_rate[0], cfg.frame_rate[1]+1, (1,))
step = frame_rate.item()
with h5py.File(path, "r") as f:
    total_frames = f["frames"].shape[0]
    n_avail = min(cfg.seq_len * step, total_frames)
    frames_np = f["frames"][0:n_avail:step][:cfg.seq_len] # 只读 10 帧 = 3.1 MB
```

```
sel = slice(0, n_avail, step)
target_point = data["target_point"][idx][sel][:cfg.seq_len]
```

实测：43.2 ms → 0.7 ms，加速 57×，零额外存储开销。

4. 综合提速预期

改动	独立预期效果	说明
BF16 AMP	2–3× 计算加速	Tensor Core BF16 算力 vs FP32
torch.compile	+20–40%	首 epoch 编译约 1 min，之后生效
batch_size 2→8	4× 吞吐	相同时间处理 4× 样本
h5 strided read	57× I/O 加速	3.1 MB vs 116 MB per sample
综合（保守估计）	5–8× 端到端加速	各项收益部分叠加

若要进一步提速：num_workers 方向

要启用 num_workers > 0，需将 optical_flow / edge 的 GPU 计算移出 Dataset。两种路径：

- 路径 A（推荐）：将 optical_flow / edge 移入 Model.train() / test()，Dataset 只返回原始帧，cfg.device=cpu，num_workers=4，pin_memory=True。
- 路径 B：将 optical_flow / edge 降级为 CPU 计算（RAFT on CPU 约 500 ms/pair，4 worker 并行可能仍比 GPU 串行慢，不推荐）。